universität
innsbruck

# Kontrollstrukturen

## Einführung in die Programmierung

Michael Tschuggnall
Universität Innsbruck

# Überblick

- Einführung
- Vorschau
- Variablen und Datentypen
- Operatoren und Ausdrücke
- Kontrollstrukturen
  - Selektionsanweisung
  - Iterationsanweisungen
  - Sprunganweisung
- Funktionen
- Arrays
- Zeiger
- Strukturen, Unions, Bitfelder
- Speicherklassen
- Dynamische Speicherverwaltung
- Präprozessor
- Modulare Programmierung
- Ein- und Ausgabe
- gcc und make

# Motivation

- Bisher wurden (abgesehen von der Einheit Vorschau) die einzelnen Zeilen eines Programms sequentiell abgearbeitet.
- Sehr oft sollen aber bestimmte Anweisungen ausgewählt, oder eine Folge von Anweisungen wiederholt werden.
- Kontrollstrukturen definieren die Reihenfolge, in der Anweisungen durchgeführt werden.

# Arten von Anweisungen

- Block-Anweisungen:
  - Alles zwischen { und } wird zu einem Block zusammengefasst.
    - Nach der rechten Klammer steht kein Semikolon!
  - Ein Block ist syntaktisch äquivalent zu einer einzelnen Anweisung.
    - Blöcke können auch ineinander geschachtelt werden.
- Ausdrucksanweisungen:
  - Aus einem Ausdruck wird eine Anweisung, wenn ein Semikolon folgt.
- Selektionsanweisungen:
  - **if-Anweisung**
  - **switch-Anweisung**
- Iterationsanweisungen:
  - **while-Schleife**
  - **do-while-Schleife**
  - **for-Schleife**
- Sprunganweisungen:
  - **break-Anweisung**
  - **continue-Anweisung**
  - return-Anweisung
  - **goto-Anweisung**

# Selektionsanweisungen

# Selektionsanweisungen

- Bei Selektionsanweisungen wird basierend auf dem Wert eines Ausdrucks aus einer Reihe von Anweisungen ausgewählt.
  - Der Kontrollfluss kann dadurch verzweigt werden.
  - Deshalb werden Selektionsanweisungen auch Verzweigungen genannt.
- Arten von Selektionsanweisungen:
  - if-Anweisung
  - switch-Anweisung

# if-Anweisung

- Allgemeine Form:

```
if (Ausdruck)
  Anweisung_1
else
  Anweisung_2
```

- Ablauf:
  - Die Bedingung `Ausdruck` wird ausgewertet.
  - Hat die Bedingung einen von 0 verschiedenen Wert (Bedingung trifft zu), dann wird `Anweisung_1` ausgeführt.
  - Hat die Bedingung den Wert 0, dann wird `Anweisung_2` ausgeführt.
- Zu beachten:
  - Der `else`-Zweig ist optional, d.h. dieser Zweig kann weggelassen werden.
  - Bei mehreren Anweisungen muss ein Block verwendet werden (siehe Beispiel).
  - `if`-Anweisungen können verschachtelt werden.

# Beispiel if-Anweisung mit Block

- Beispiel ohne Block (schlecht formatiert und eventuell fehlerhaft):

```
1  if (value > 10)
2      printf("Statement1\n");
3      printf("Statement2\n");
```

| **Ausgabe auf der Standardausgabe für `value` ≤ `10`:** | **Ausgabe auf der Standardausgabe für `value` > `10`:** |
|---|---|
| `Statement2` | `Statement1`<br>`Statement2` |

- Beispiel mit Block:

```
1  if (value > 10) {
2      printf("Statement1\n");
3      printf("Statement2\n");
4  }
```

| **Ausgabe auf der Standardausgabe für `value` ≤ `10`:** | **Ausgabe auf der Standardausgabe für `value` > `10`:** |
|---|---|
| | `Statement1`<br>`Statement2` |

# if-Anweisung mit mehrfacher-Selektion

- Allgemeine Form:

```
if (Ausdruck_1)
  Anweisung_1
else if (Ausdruck_2)
  Anweisung_2
…
else if (Ausdruck_n)
  Anweisung_n
else
  Anweisung_else
```

- Ablauf:
  - Die Bedingungen werden von oben nach unten ausgewertet.
  - Bei der ersten Bedingung, die wahr ist, wird die zugehörige Anweisung abgearbeitet und die Mehrfach-Selektion abgebrochen.
- Zu beachten:
  - Der `else`-Zweig ist optional, d.h. dieser Zweig kann weggelassen werden.

# Dangling else

- Ein `else`-Zweig bei zwei `if`-Anweisungen.
  - Wohin gehört der `else`-Zweig?
- Beispiel (schlecht formatiert):

```
1 if (counter < 5)
2     if (counter % 2 == 0)
3         printf("HERE1");
4 else
5     printf("HERE2");
```

- Auflösung durch Compiler:
  - `else`-Zweig gehört zum textuell letzten freien `if` im selben Block!
  - `counter` mit Wert `7` führt zu keiner Ausgabe.
  - `counter` mit Wert `3` führt zur Ausgabe `"HERE2"`.
- Empfehlung:
  - Für if-Anweisungen und Iterationsanweisungen immer Blöcke verwenden!

# switch-Anweisung (1)

- Allgemeine Form:

```
switch (Ausdruck_1) {
    case k1:
        Anweisungen_1
        break;
    case k2:
        Anweisungen_2
        break;
    …
    case kn:
        Anweisungen_n
        break;
    default:
        Anweisungen_default
        break;
}
```

# switch-Anweisung (2)

- Die Anweisungen `Anweisungen_1`, ..., `Anweisungen_n` sowie `Anweisungen_default` sind die Alternativen der switch-Anweisung.
- Die case-Marken und die optionale default-Marke markieren die Einstiegspunkte der Alternativen.
- Nach jeder case-Marke muss ein konstanter ganzzahliger Ausdruck angegeben werden!
  - Der Wert von jedem dieser Ausdrücke muss unterschiedlich sein.

# switch-Anweisung (3)

- Hat der Ausdruck der switch-Anweisung den gleichen Wert wie einer der konstanten Ausdrücke der case-Marken, dann wird die Ausführung des Programms mit der Anweisung (den Anweisungen) hinter dieser case-Marke weitergeführt.
  - Nach der Abarbeitung der Anweisungen wird die switch-Anweisung verlassen, falls eine break-Anweisung vorhanden ist.
  - Fehlt die break-Anweisung, werden alle nachfolgenden Anweisungen bis zur nächsten break-Anweisung oder dem Ende der switch-Anweisung abgearbeitet („fall through“).
- Stimmt keiner der konstanten Ausdrücke der case-Marken mit dem switch-Ausdruck überein:
  - Es wird zur default-Marke gesprungen, falls vorhanden.
  - Wenn keine default-Marke vorhanden ist, wird die Anweisung nach der switch-Anweisung ausgeführt.

# Beispiel switch-fall through

example_switch_fallthrough.c

```c
#include <stdio.h>
#include <stdlib.h>
enum weekday { MON = 1, TUE, WED, THU, FRI, SAT, SUN };
int main(void) {
    enum weekday day = THU;
    switch (day) {
        case MON:
        case TUE:
        case WED:
        case THU:
        case FRI:
            printf("business day\n");
            break;
        case SAT:
        case SUN:
            printf("weekend\n");
            break;
    }
    return EXIT_SUCCESS;
}
```

# Aufgabe Transformation

- Formulieren Sie die nachfolgende if-Anweisung in eine switch-Anweisung um!

```c
if (option == 1) {
    printf("option 1\n");
} else if (option == 2) {
    printf("option 2\n");
} else {
    printf("unknown option!\n");
}
```

# Aufgabe Transformation - Lösung

- Formulieren Sie die nachfolgende if-Anweisung in eine switch-Anweisung um!

```c
switch (option) {
    case 1:
        printf("option 1\n");
        break;
    case 2:
        printf("option 2\n");
        break;
    default:
        printf("unknown option!\n");
        break;
}
```

# switch- und if-Anweisung im Vergleich

- Die switch-Anweisung prüft nur auf die Gleichheit von Werten im Gegensatz zur if-Anweisung, bei der Bedingungen ausgewertet werden.
- Die case-Marken der switch-Anweisung können nur ganzzahlige konstante Ausdrücke verarbeiten.
- Die switch-Anweisung ist übersichtlicher und leichter zu erweitern.

# Iterationsanweisungen

# Iterationsanweisungen

- Durch Iterationsanweisungen kann eine Anweisung, welche als Schleifenkörper bezeichnet wird, wiederholt ausgeführt werden.
- Arten von Iterationsanweisungen:
  - while-Anweisung
  - do-while-Anweisung
  - for-Anweisung
- Diese Anweisungen werden auch als Schleifen bezeichnet.

# while-Schleife

- Allgemeine Form:

```
while (Ausdruck)
   Anweisung
```

- Ablauf:
  1. Die Bedingung `Ausdruck` wird ausgewertet.
  2. Ist die Bedingung ungleich 0:
     - Der Schleifenkörper (`Anweisung`) wird ausgeführt.
     - Nach der Ausführung: Gehe zu Schritt 1.
  3. Ist die Bedingung gleich 0, wird die nächste Anweisung nach der Schleife ausgeführt.
- Wichtig:
  - Beliebiger Teil des Ausdrucks muss im Schleifenrumpf (`Anweisung`) verändert werden (ansonsten liegt eine Endlosschleife vor).
  - Bei mehreren Anweisungen muss ein Block verwendet werden.

# Beispiel while-Schleife

example_while.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 0;
    while (i <= 10) {
        printf("%d", i);
        if (i < 10) {
            printf(", ");
        }
        ++i;
    }
    return EXIT_SUCCESS;
}

```

**Ausgabe auf der Standardausgabe:**

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

# Aufgabe while-Schleife

- Das nachfolgende Codefragment soll alle Ganzzahlen von 0 bis 9 (inklusive der Grenzen) in aufsteigender Reihenfolge untereinander ausgeben. Identifizieren Sie alle Fehler und korrigieren Sie diese!

exercise_while.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 0;
    while (i > 10)
        printf("%d\n", i);
        i++;

    return EXIT_SUCCESS;
}
```

# Aufgabe while-Schleife – Lösung

- Das nachfolgende Codefragment soll alle Ganzzahlen von 0 bis 9 (inklusive der Grenzen) in aufsteigender Reihenfolge untereinander ausgeben. Identifizieren Sie alle Fehler und korrigieren Sie diese!

exercise_while.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 0;
    while (i < 10) {
        printf("%d\n", i);
        i++;
    }
    return EXIT_SUCCESS;
}
```

# for-Schleife

- Allgemeine Form:

```
for (Ausdruck_1; Ausdruck_2; Ausdruck_3)
   Anweisung
```

- Aufbau:
  - Ausdruck_1: Initialisierung
    - Einmalige Initialisierung von Variablen (z.B. einer Laufvariable, welche die Anzahl der Schleifendurchläufe zählt) vor dem ersten Durchlauf der Schleife.
    - Schleifenvariablen können auch im ersten Ausdruck deklariert werden.
  - Ausdruck_2: Schleifenbedingung
  - Ausdruck_3: Anpassung
    - Verändern der Werte von Variablen (z.B. Laufvariable erhöhen).
- Ablauf:
  1. Initialisierung wird ausgeführt.
  2. Die Schleifenbedingung wird ausgewertet.
  3. Ist die Schleifenbedingung ungleich 0:
     - Der Schleifenkörper (`Anweisung`) wird ausgeführt.
     - Die Anpassung wird ausgeführt.
     - Nach der Ausführung: Gehe zu Schritt 2.
  4. Ist die Schleifenbedingung gleich 0, wird die nächste Anweisung nach der Schleife ausgeführt.

# Beispiel for-Schleife

example_for.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    for (int i = 0; i <= 10; ++i) {
        printf("%d", i);
        if (i < 10) {
            printf(", ");
        }
    }
    return EXIT_SUCCESS;
}
```

**Ausgabe auf der Standardausgabe:**

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

# Aufgabe for-Schleife

- Auf den ersten Blick scheint bei der nachfolgenden for-Schleife alles in Ordnung zu sein. Auch logisch liegt kein Fehler vor. Warum terminiert diese Schleife nicht und was können Sie dagegen tun?

exercise_for.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    for (float i = 0.0f; i != 1.0f; i += 0.1f) {
        printf("%f\n", i);
    }
    return EXIT_SUCCESS;
}
```

# Aufgabe for-Schleife – Lösung

- Auf den ersten Blick scheint bei der nachfolgenden for-Schleife alles in Ordnung zu sein. Auch logisch liegt kein Fehler vor. Warum terminiert diese Schleife nicht und was können Sie dagegen tun?

exercise_for.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    for (int i = 0; i < 10; ++i) {
        printf("%f\n", i / 10.0f);
    }
    return EXIT_SUCCESS;
}
```

# Komma-Operator

- Es ist auch möglich, im Schleifenkopf einer for-Schleife mehrere Anweisungen mit dem Komma-Operator getrennt zu verwenden.
- Dies wird zum Beispiel dazu verwendet, im ersten Ausdruck mehrere Variablen mit einem Wert zu initialisieren und/oder im dritten Ausdruck mehrere Variablen zu verändern.

# Beispiel for-Schleife mit Komma-Operator

example_for_comma.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    for (int i = 0, j = 9; i < j; ++i, --j) {
        printf("i = %d, j = %d\n", i, j);
    }
    return EXIT_SUCCESS;
}
```

**Ausgabe auf der Standardausgabe:**

```
i = 0, j = 9
i = 1, j = 8
i = 2, j = 7
i = 3, j = 6
i = 4, j = 5
```

# Beispiel geschachtelte for-Schleife

example_for_nested.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    for (int i = 1; i < 4; i++) {
        for (int j = 1; j < 4; j++) {
            printf("%d * %d = %d\n", i, j, i * j);
        }
    }
    return EXIT_SUCCESS;
}

```

**Ausgabe auf der Standardausgabe:**

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
```

# for-Schleife (Spezialfälle)

- Jeder der drei Ausdrücke kann entfallen.
  - Fehlt `Ausdruck_1`, dann entfällt die Initialisierung.
  - Fehlt `Ausdruck_2`, so ist die Bedingung immer wahr (Endlosschleife).
  - Fehlt `Ausdruck_3`, so fehlt die Anpassung.
- Abbruch einer Endlosschleife:
  - Die Schleife kann dann durch eine break-Anweisung abgebrochen werden.
    - Sollte nur in Ausnahmefällen verwendet werden.
- Leere Anweisung:
  - Soll keine Anweisung ausgeführt werden, dann muss trotzdem ein einzelner Strichpunkt oder ein leerer Block angegeben werden.
  - `for(…);` oder `for(…){}`

# while- oder for-Schleife?

- Wann ist eine for- und wann eine while-Schleife zu verwenden ?
  - for-Schleifen werden üblicherweise dann verwendet, wenn ein ganzer Bereich beginnend bei einem bestimmten Startwert bis zu einem vorgegebenen Endwert mit einer festen Schrittweite zu durchlaufen ist.
  - while-Schleifen werden typischerweise verwendet, wenn lediglich ein Endkriterium gegeben ist, und es nicht vorhersagbar ist, wie oft die Schleifenanweisungen ausgeführt werden müssen, bis das Endkriterium zutrifft.
  - Im Allgemeinen sollte die im vorliegenden Kontext besser lesbare Variante gewählt werden.
- Jede for-Schleife lässt sich durch eine while-Schleife formulieren und umgekehrt!

| for-Schleife | entsprechende while-Schleife |
|---|---|
| **for** (*Ausdruck_1*; *Ausdruck_2*; *Ausdruck_3*)<br>  Anweisung; | *Ausdruck_1*;<br>**while** (*Ausdruck_2*) {<br>  Anweisung;<br>  *Ausdruck_3*;<br>} |

# Aufgabe Transformation

- Was wird durch das folgende Programm ausgegeben? Formulieren Sie die while-Schleife als for-Schleife!

exercise_transform_for.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int sum = 0;
    int i = 0;
    while (i <= 5) {
        if (i % 2) {
            sum += i;
        }
        i++;
    }
    printf("sum = %d\n", sum);
    return EXIT_SUCCESS;
}
```

# Aufgabe Transformation – Lösung (1)

- Was wird durch das folgende Programm ausgegeben? Formulieren Sie die while-Schleife als for-Schleife!

exercise_transform_for.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int sum = 0;
    for (int i = 0; i <= 5; ++i) {
        if (i % 2) {
            sum += i;
        }
    }
    printf("sum = %d\n", sum);
    return EXIT_SUCCESS;
}
```

**Ausgabe auf der Standardausgabe:**

```
sum = 9
```

# Aufgabe Transformation – Lösung (2)

- Was wird durch das folgende Programm ausgegeben? Formulieren Sie die while-Schleife als for-Schleife!

exercise_transform_for.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int sum = 0;
    for (int i = 1; i <= 5; i += 2) {
        sum += i;
    }
    printf("sum = %d\n", sum);
    return EXIT_SUCCESS;
}
```

# do-while-Schleife

- Allgemeine Form:

```
do {
   Anweisung
} while (Ausdruck);
```

- Ablauf:
  1. Der Schleifenkörper (`Anweisung`) wird ausgeführt (annehmende Schleife).
  2. Die Bedingung `Ausdruck` wird ausgewertet.
  3. Ist die Bedingung ungleich 0, gehe zu Schritt 1.
  4. Ist die Bedingung gleich 0, wird die nächste Anweisung nach der Schleife ausgeführt.
- Hinweis:
  - Am Ende steht ein Strichpunkt, da hier das Ende der Anweisung erreicht wird.

# Beispiel do-while-Schleife

example_do_while.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int value = 12345;
    unsigned char digits = 0;
    do {
        ++digits;
        value /= 10;
    } while (value);
    printf("digits = %hhu\n", digits);
    return EXIT_SUCCESS;
}
```

**Ausgabe auf der Standardausgabe:**

```
digits = 5
```

# Endlosschleifen

- Schleifen, bei denen kein Abbruchkriterium definiert wurde bzw. das Abbruchkriterium immer wahr ist.
- Typische Erstellungsmöglichkeiten von Endlosschleifen:
  - `while(1) {…}`
  - `for(;;) {…}` oder `for(;1;) {…}`
- Beendigung von Endlosschleifen:
  - Sprung mit `break` aus Endlosschleife
  - Beendigung einer Funktion, in der eine Endlosschleife läuft, mit `return`
  - Beendigung eines Programms mit `exit`

# Sprunganweisungen

# Sprunganweisungen

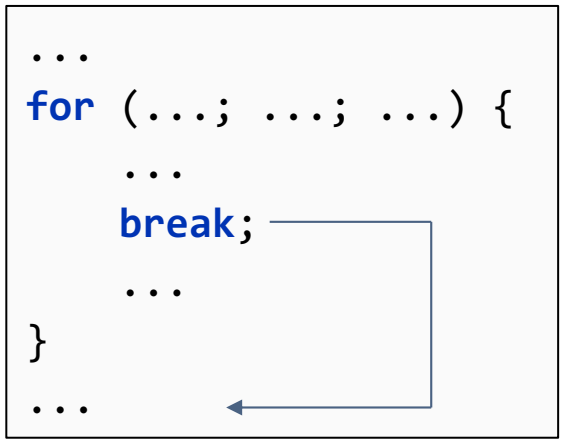
- Durch Sprunganweisungen wird zu einem anderen Ort gesprungen und die Ausführung dort weitergeführt.
- Arten von Sprunganweisungen:
  - break-Anweisung
  - continue-Anweisung
  - goto-Anweisung
  - return-Anweisung (siehe Foliensatz zu Funktionen)

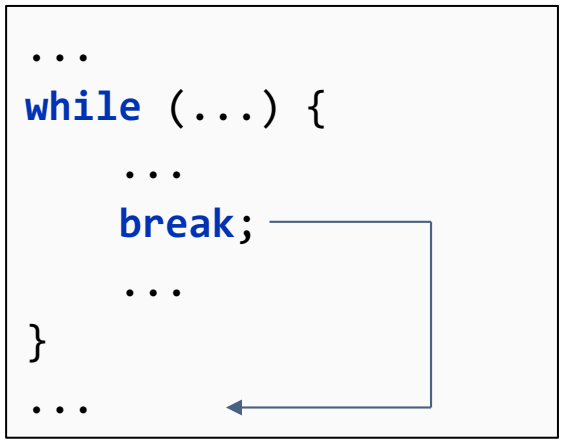
# break- und continue-Anweisung

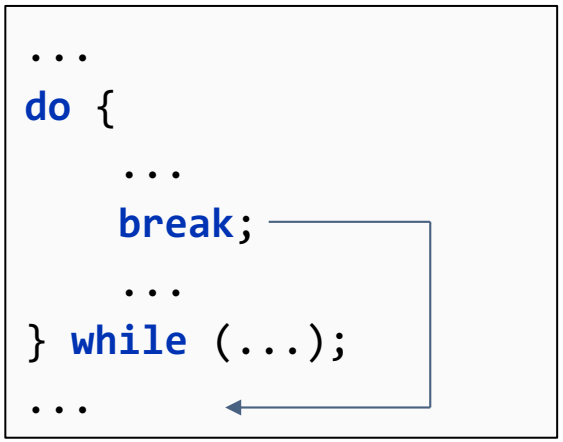
- break-Anweisung:
  - Wird verwendet, um die Ausführung der umgebenden switch-Anweisung oder Schleife zu beenden.
  - Darf nur in Schleifenkörpern oder in switch-Anweisungen verwendet werden.
- continue-Anweisung:
  - Wird verwendet, um den Rest der Anweisungsfolge des Schleifenkörpers der umgebenden Schleife zu überspringen.
  - Darf nur im Schleifenkörper verwendet werden.

- Verwendung:
  - Bei switch-Anweisungen ist die break-Anweisung meist notwendig!
  - In Schleifen nur sparsam verwenden!
    - Kann übermäßige Verschachtelung vermeiden!
    - Kann durch entsprechende Formulierung der Schleifenbedingung umgangen werden!

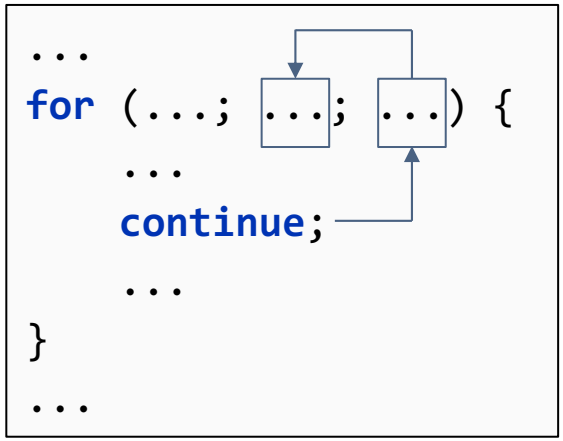
# Veranschaulichung break

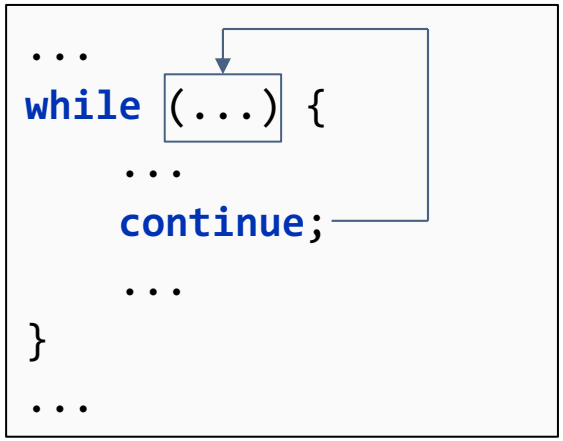


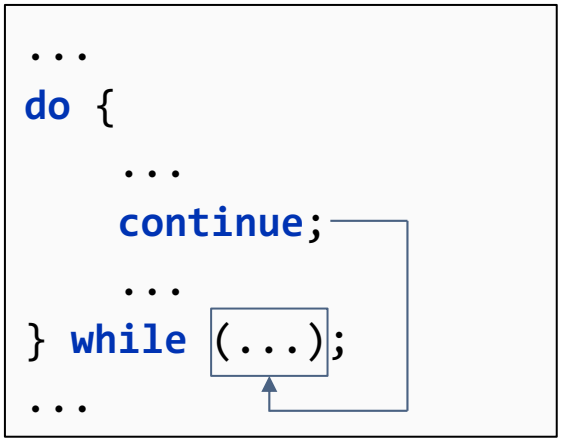



# Veranschaulichung continue



# Aufgabe continue

- Warum terminiert das folgende Programm nicht? Nehmen Sie eine entsprechende Korrektur vor, damit das Programm die ungeraden Zahlen von 0 bis 20 in aufsteigender Reihenfolge untereinander ausgibt!

exercise_continue.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 0;
    while (i < 20) {
        if (i % 2) {
            continue;
        }
        printf("%d\n", i);
        i++;
    }
    return EXIT_SUCCESS;
}
```

# Aufgabe continue – Lösung (1)

- Warum terminiert das folgende Programm nicht? Nehmen Sie eine entsprechende Korrektur vor, damit das Programm die ungeraden Zahlen von 0 bis 20 in aufsteigender Reihenfolge untereinander ausgibt!

exercise_continue.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 0;
    while (i < 20) {
        if (i % 2) {
            printf("%d\n", i);
        }
        i++;
    }
    return EXIT_SUCCESS;
}
```

# Aufgabe continue – Lösung (2)

- Warum terminiert das folgende Programm nicht? Nehmen Sie eine entsprechende Korrektur vor, damit das Programm die ungeraden Zahlen von 0 bis 20 in aufsteigender Reihenfolge untereinander ausgibt!

exercise_continue.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    for (int i = 1; i < 20; i += 2) {
        printf("%d\n", i);
    }
    return EXIT_SUCCESS;
}
```

# goto-Anweisung

- Allgemeine Form:

```
goto Sprungmarke;
…
Sprungmarke : Anweisung
```

- Diese Anweisung bewirkt einen Sprung zu der Programmzeile, die mit einer Sprungmarke markiert wurde.
- Nur in **ganz wenigen** Ausnahmefällen sinnvoll!
  - Jedes Programm kann auch ohne `goto` realisiert werden!

# „Theoretisches Beispiel“ goto

example_goto.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (i + j == 4) {
                printf("jump to breakout\n");
                goto breakout;
            }
            printf("%d %d\n", i, j);
        }
    }
breakout:
    printf("after breakout!\n");
    return EXIT_SUCCESS;
}
```

**Ausgabe auf der Standardausgabe:**

```
0 0
0 1
0 2
0 3
1 0
1 1
1 2
jump to breakout
after breakout!
```